

Use Azure Speech in an agent

[Prerequisites](#)

[Create an Azure storage account](#)

[Create a Microsoft Foundry project](#)

[Create an agent](#)

[Create an Azure Speech in Foundry Tools connection](#)

[Test the Azure Speech tool in the playground](#)

[Create a client application](#)

[Test the client application](#)

[Clean up resources](#)

Azure Speech in Foundry Tools provides an MCP server that you can use to enable an agent to call its speech recognition and synthesis capabilities.

In this exercise, you'll configure the Azure Speech in Foundry Tools MCP server, and connect it to an agent.

The code used in this exercise is based on the Microsoft Foundry SDK for Python. You can develop similar solutions using the SDKs for Microsoft .NET, JavaScript, and Java. Refer to [Microsoft Foundry SDK client libraries](#) for details.

This exercise takes approximately **30** minutes.

Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](#)
- [Visual Studio Code](#) installed
- [Python version 3.13.xx](#) installed*
- [Git](#) installed and configured
- [Azure CLI](#) installed

* Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create an Azure storage account

The Azure Speech MCP server uses an Azure storage account to save generated audio files.

1. Open the [Azure portal](#) at `https://portal.azure.com` , and sign in using your Azure credentials.
2. Create a new **Azure storage account** resource with the following settings:
 - **Subscription:** *Your subscription*
 - **Resource group:** *Create or select a resource group*
 - **Storage account name:** *A unique name for your storage account*
 - **Region:** *Any available region*
 - **Preferred storage type:** Azure blob storage or Azure Data Lake Storage Gen2
 - **Primary workload:** Cloud native
 - **Performance:** Standard
 - **Redundancy:** Locally-redundant storage (LRS)
3. When the Azure storage account resource has been created, go to it in the portal.
4. In the left navigation pane for the storage account, expand **Data storage**, and select **Containers**.
5. Add a new container named **files**. This is where your agent will save the audio files it generates.
6. In the context menu (...) for the **files** container, select **Generate SAS**, and create a SAS token with the following details:
 - **Signing method :** Account key
 - **Signing key:** Key1
 - **Stored access policy:** None

- **Permissions:**
 - Read
 - Add
 - Create
 - Write
 - List
- **Start and expiry date/time:**
 - **Start:** The current date and time
 - **Expiry:** 11:59pm tomorrow
- **Allowed IP addresses:** *Leave blank*
- **Allowed protocols:** HTTPS only

! **IMPORTANT:** Copy the generated SAS token and URL, and store them in a text file for now - you'll need them later!

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open the [Microsoft Foundry portal](#) at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the Foundry logo at the top left to navigate to the home page.
2. If it is not already enabled, in the tool bar the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
 - **Foundry resource:** *Use the default name for your resource (usually {project_name}-resource)*
 - **Subscription:** *Your Azure subscription*
 - **Resource group:** *Create or select a resource group*
 - **Region:** Select any available region

! **Note:** Use a recommended Microsoft Foundry region. Model availability may vary by region.

! **TIP:** Remember (or make a note of) the Foundry resource name - you're going to need it later!

3. Wait for your project to be created.
4. On the home page for your project, note that the API key, project endpoint, and OpenAI endpoint are displayed here.

! **TIP:** Copy the project key to the clipboard - you're going to need it later!

Create an agent

Now that you have a Foundry project, you can create an agent.

1. Now you're ready to **Start building**. Select **Create agents** (or on the **Build** page, select the **Agents** tab); and create a new agent named `speech-agent`.
- When ready, your agent opens in the agent playground.
2. In the model drop-down list, ensure that a **gpt-5** model has been deployed and selected for your agent.
3. Assign your agent the following **Instructions**:

CodeCopy

You are an AI agent that uses the Azure AI Speech tool to transcribe and generate speech.

- 4. Use the **Save** button to save the changes.
- 5. Test the agent by entering the following prompt in the **Chat** pane:

CodeCopy

What can you help me with?

The agent should respond with an appropriate answer based on its instructions.

Create an Azure Speech in Foundry Tools connection

Foundry includes an MCP server for Azure Speech in Foundry Tools, which you can connect to your project and use in your agent.

- 1. In the navigation pane on the left, select the **Tools** page.
- 2. On the **Tools** tab, connect a tool; selecting **Azure Speech MCP Server** in the **Catalog** and connecting it to an endpoint. specifying the following configuration
 - o **Name:** *A unique name for your tool.*
 - o **Remote MCP Server endpoint:**

https://{foundry-resource-name}.cognitiveservices.azure.com/speech/mcp?api-version=2025-11-15-preview
 - o **Parameters:** foundry-resource-name: *Your foundry resource name*
 - o **Authentication:** Key-based:
 - o **Ocp-Apim-Subscription-Key:** *API Key for your Foundry project*
 - o **X-Blob-Container-Url:** *The SAS URL for your storage container*

Note: If key-based authentication is disabled by a policy in your Azure subscription, you can use Entra ID authentication to connect the agent to the Azure Speech service.

- 3. Wait for the MCP tool connection to be created, and then view its details page.
- 4. On the details page for the Azure Speech in Foundry Tools connection, select **Use in an agent**, and then select the **Speech-Agent** agent you created previously.

The agent should open in the playground, with the Azure Speech in Foundry Tools tool connected.

Test the Azure Speech tool in the playground

Now let's test the agent's ability to use the tool you connected.

- 1. In the agent playground for the **speech-agent** agent, enter the following prompt:

CodeCopy

Generate "To be or not to be, that is the question." as speech

- 2. When prompted, approve use of the Azure Speech tool by selecting **Always approve all Azure Speech MCP Server tools**.

3. Review the response, which should include a link to the generated audio file. Then click the link to hear the synthesized speech.
4. Enter the following prompt:

Code

Copy

Transcribe the file at https://microsoftlearning.github.io/mslearn-ai-language/Labfiles/05-speech-tool/speech_1.wav

5. If prompted, approve use of the Azure Speech tool by selecting **Always approve all Azure Speech MCP Server tools**.

!

Note: If a transient error occurs while calling the tool, refresh the page and retry the same prompt.

!

Note: You may see a message stating that the interaction was blocked by a Foundry guardrail. This happens because Foundry's built-in content safety checks flag the transcribed content of `speech_1.wav`. If this occurs, retry the prompt using `speech_2.wav` instead, which is not affected.

6. Review the output, which should be a transcription of the audio file.

Create a client application

Now that you have a working agent, you can create a client application that uses it.

Get the application files from GitHub

1. Open Visual Studio Code.
2. Open the command palette (*Ctrl+Shift+P*) and use the `Git:clone` command to clone the `https://github.com/microsoftlearning/mslearn-ai-language` repo to a local folder (it doesn't matter which one). Then open it.
- You may be prompted to confirm you trust the authors.
3. After the repo has been cloned, in the Explorer pane, navigate to the folder containing the application code files at **/Labfiles/05-speech-tool/Python/speech-client**. The application files include:
- `.env` (the application configuration file)
 - `requirements.txt` (the Python package dependencies that need to be installed)
 - `speech-client.py` (the code file for the application)

Configure the application

1. In Visual Studio Code, view the **Extensions** pane; and if it is not already installed, install the **Python** extension.
2. In the **Command Palette**, use the command `python:select interpreter`. Then select an existing environment if you have one, or create a new **Venv** environment based on your Python 3.1x installation.

!

Tip: If you are prompted to install dependencies, you can install the ones in the `requirements.txt` file in the `/Labfiles/05-speech-tool/Python/speech-client` folder; but it's OK if you don't - we'll install them later!

!

Tip: If you prefer to use the terminal, you can create your **Venv** environment with `python -m venv labenv`, then activate it with `\labenv\Scripts\activate`.

3. In the **Explorer** pane, right-click the **speech-client** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the `/Labfiles/05-speech-tool/Python/speech-client` folder.)

!

Note: Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

4. Ensure that the terminal is open in the **speech-client** folder with the prefix **(.venv)** to indicate that the Python environment you created is active.
5. Install the Foundry SDK package, the Azure Identity package, and other required packages by running the following command:

Code

Copy

```
pip install -r requirements.txt
```

6. In the **Explorer** pane, in the **speech-client** folder, select the **.env** file to open it. Then update the configuration values to include your project **endpoint** (from the project home page in Foundry Portal) and the name of your agent (which should be **speech-agent** - note that this name is case-sensitive).
7. Save the modified configuration file.

Implement application code

1. In the **Explorer** pane, in the **speech-client** folder, open the **speech-client.py** file.
2. Review the existing code. You will add code to submit prompts to your agent.

!

Tip: As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespaces you will need:

Code

Copy

```
# import namespaces
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
```

4. In the **main** function, note that code to load the endpoint from the configuration file has already been provided. Then find the comment **Get project client**, and add the following code to create a client for your Foundry project:

Code

Copy

```
# Get project client
project_client = AIProjectClient(
    endpoint=foundation_endpoint,
    credential=DefaultAzureCredential(),
)
```

5. Find the comment **Get an OpenAI client**, and add the following code to get an OpenAI client with which to call your agent.

Code

Copy

```
# Get an OpenAI client
openai_client = project_client.get_openai_client()
```

- 6. Find the comment **Use the agent to get a response**, and add the following code to submit a user prompt to your agent, and display the response.

CodeCopy

```
# Use the agent to get a response
response = openai_client.responses.create(
    input=[{"role": "user", "content": prompt}],
    extra_body={"agent_reference": {"name": agent_name, "type": "agent_reference"}},
)

print(f"{agent_name}: {response.output_text}")
```

- 7. Save the changes you made to the code file.

Test the client application

Now let's test the application by running it in a Python environment and authenticating the connection to your project.

- 1. In the terminal pane, use the following command to sign into Azure.

CodeCopy

```
az login
```

!

Note: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *--tenant* parameter. See [Sign into Azure interactively using the Azure CLI](#) for details.

- 2. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.
- 3. After you have signed in, enter the following command to run the application:

CodeCopy

```
python speech-client.py
```

- 4. When prompted, enter the following prompt:

CodeCopy

```
Synthesize "Better a witty fool, than a foolish wit!" as speech using the voice "en-GB-SoniaNeural".
```

- 5. Review the response, which should include a clickable link to a generated audio file.
- 6. After checking out the generated audio file, enter the following prompt:

CodeCopy

```
Transcribe https://microsoftlearning.github.io/mslearn-ai-language/Labfiles/05-speech-tool/speech_2.wav
```

7. Review the response.
8. To exit the program, enter "quit" (or just press return)

Clean up resources

If you're finished exploring the Azure AI Language service, you can delete the resources you created in this exercise. Here's how:

1. In the Azure portal, browse to the Foundry resource you created in this lab.
2. On the resource page, select **Delete** and follow the instructions to delete the resource.